
UNIT 4 DATA ACQUISITION (DAQ): PROCESS

Structure

- 4.0 Introduction
- 4.1 Objectives
- 4.2 DAQ Process:
 - 4.2.1 Sensor Integration
 - 4.2.2 Signal Conditioning
 - 4.2.3 Multiplexing (if needed)
 - 4.2.4 Triggering (if applicable)
 - 4.2.5 Sampling
 - 4.2.6 ADC (Analog-to-Digital Conversion)
 - 4.2.7 Data Processing
 - 4.2.8 Data Logging
 - 4.2.9 Display/Control/Transmission
- 4.3 DAC (Digital-to-Analog Conversion),
- 4.4 High Speed DAQ,
- 4.5 Summary
- 4.6 Solutions/Answers
- 4.7 Further Readings

4.0 INTRODUCTION

In **data science**, **data acquisition** refers to the process of collecting data from various sources for analysis. The quality and accuracy of the data play a crucial role in the success of data science projects, as the subsequent analysis, model building, and decision-making depend on it. Data can be collected from a variety of techniques, including sensors, web scraping, APIs, surveys, and existing databases. In our Unit-3 we learned about the tools of data acquisition, here in this unit we are going to extend our discussion towards the process of data acquisition.

The process of data acquisition is comprised of various stages, mentioned below:

1. Sensor Integration
2. Signal Conditioning
3. Multiplexing (if needed)
4. Triggering (if applicable)
5. Sampling
6. ADC (Analog-to-Digital Conversion)
7. Data Processing

8. Data Logging
9. Display/Control/Transmission

This order ensures the smooth flow of data from physical signals to digital information and subsequent storage for analysis in data science applications.

4.1 OBJECTIVES

After finishing this unit, you will be able to:

- Understand the step-by-step process involved in data acquisition systems.
- Learn how sensors are integrated and configured for accurate data collection.
- Explore how signal conditioning prepares raw signals for digital conversion.
- Understand the roles of sampling, ADC, and data processing in DAQ.
- Gain insight into data logging, control, display, and transmission in DAQ systems.

4.2 DAQ: PROCESS

The **Data Acquisition (DAQ)** process refers to the method of collecting and measuring physical phenomena—such as temperature, pressure, voltage, or motion—and converting them into digital data that can be analysed by a computer. We had already explored the DAQ tools in unit – 3, and in this section, we are going to discuss the entire process of Data Acquisition, but prior to that a short introduction of the stages of the data acquisition process is presented below, to establish a better connection with the remaining sections of this unit.

Stages of Data Acquisition:

1. **Sensor Integration:** The first step involves integrating the sensors into the data acquisition system. Sensors detect real-world physical phenomena (e.g., temperature, pressure, motion) and convert them into electrical signals (usually analog). These sensors need to be connected to the data acquisition system, and this step ensures proper communication between the sensors and the system.
2. **Signal Conditioning:** Once the raw signal from the sensor is obtained, it often requires conditioning. Signal conditioning may include amplification, filtering, and conversion (e.g., adjusting the signal strength or removing noise). The goal is to make the sensor's output compatible with the input requirements of the Analog-to-Digital Converter (ADC). This step is essential for preparing the signal for accurate sampling and processing.
3. **Multiplexing** (if required): In cases where multiple sensors or channels are involved, **multiplexing** allows signals from different sensors to be sent through a single ADC by switching between them in a time-

multiplexed manner. This ensures that all sensors can be read without requiring separate ADCs for each sensor.

4. **Triggering** (if applicable): **Triggering** ensures that the data acquisition system starts recording only when certain conditions are met. It helps to capture specific events, such as a voltage threshold being crossed or a particular time instance. Triggering can be applied at the start of the sampling process to ensure that important events are captured precisely.
5. **Sampling**: After signal conditioning (and possibly multiplexing), the conditioned signal is sampled at discrete intervals in time. Sampling converts the continuous signal into discrete data points at specific time intervals, which are necessary for digital processing. The **sampling rate** should be chosen to meet the **Nyquist criterion** to avoid aliasing.

Note: The **Nyquist criterion** is a fundamental principle in signal processing that states:

To accurately reconstruct a continuous signal from its samples, the sampling rate must be at least twice the highest frequency present in the signal.

This minimum required sampling rate is known as the **Nyquist rate**. The Nyquist criterion prevents **aliasing**, which is a distortion that occurs when a signal is undersampled and different signals become indistinguishable from each other in the sampled data.

6. **ADC (Analog-to-Digital Conversion)**: The sampled analog signal is converted into a digital signal through the Analog-to-Digital Converter (ADC). The ADC transforms the analog values into digital data that can be understood and processed by the data acquisition system, such as a computer or microcontroller.
7. **Digital Processing**: After the ADC converts the signal into binary, the resulting data may undergo further processing in a microcontroller or digital signal processor (DSP). This processing might include scaling, filtering, or error correction to refine the output data.
8. **Data Logging**: It involves saving the acquired data in a storage medium such as a hard drive, cloud storage, or local memory for later analysis. This step ensures the data is available for real-time or post-processing analysis.
9. **DAC (Digital-to-Analog Conversion)** (if applicable, for output or feedback systems): If there is a need to output the processed data back into an analog format (e.g., for controlling a device), **Digital-to-Analog Conversion (DAC)** is performed. This step is not always necessary in standard data acquisition systems unless there is a need for feedback control or analog signal output.

Now, we are going to discuss the above-mentioned stages of the process of Data Acquisition, in the remaining part of this unit. The first stage of DAQ is sensor integration and it is discussed in the subsequent section.

1. What is data acquisition in the context of data science?

.....

.....

.....

2. What types of sources can data be collected from in data acquisition?

.....

.....

.....

3. What is the main purpose of the Data Acquisition (DAQ) process?

.....

.....

.....

4. Why is signal conditioning necessary before sampling in a DAQ system?

.....

.....

.....

5. What is the role of triggering in the data acquisition process?

.....

.....

.....

4.2.1 Sensor Integration

In any Data Acquisition (DAQ) system, the Sensor integration refers to the process of connecting and configuring sensors with the DAQ hardware and software to ensure accurate and efficient data collection. In data science, the purpose of sensor integration is to enable the reliable and real-time collection of structured, high-quality data from the physical world, which can then be used for analysis, modelling, and decision-making. This step of the DAQ process involves both the physical connection (e.g., wiring, mounting) and logical configuration (e.g., calibration, scaling, and signal compatibility), it is a crucial step as it directly affects the data.

It is the stage wherein one needs to assure that data is correctly collected with minimum errors and distortion, further the compatibility of the signals received from sensors need to be assured and accordingly calibration is required to be performed. Some of the key reasons behind the utility of this stage of data acquisition are mentioned below:

Key reasons of proper Sensor Integration:

1. **Accurate Data Collection:** Proper integration ensures that the sensor's output is correctly interpreted by the DAQ system, minimizing errors and distortions.
2. **Signal Compatibility:** Sensors output different signal types (e.g., voltage, current, resistance, digital pulses). Integration includes matching these with suitable signal conditioning and input channels.
3. **Calibration and Scaling:** Sensors often need to be calibrated so their outputs correspond accurately to real-world units (e.g., mV to °C). Integration ensures this translation is handled correctly in the software.
4. **System Reliability:** Good integration minimizes noise, interference, or data loss, making the system more robust for long-term operation.
5. **Automation and Control:** Integrated sensors can be used in feedback loops for automation, enabling real-time decision-making or system adjustments.

In essence, sensor integration is a critical step to make sure that the physical signals are accurately and efficiently turned into usable digital data by the DAQ system

A structured explanation of the steps involved in sensor integration is given below:

1. **Define the Application Requirements:** Before selecting and integrating any sensor, it's essential to clearly define what you need to measure, the environmental conditions, the expected input/output range, and the communication protocol requirements. This ensures that the sensor you choose aligns with the functional goals and constraints of the application.

Example: In a smart agriculture project, the goal is to monitor soil moisture to automate irrigation. So, the requirement is a soil moisture sensor with digital output compatible with a microcontroller (e.g., Arduino) operating in outdoor conditions.

2. **Select the Appropriate Sensor:** Based on the application requirements, choose a sensor that fits the necessary parameters like sensitivity, accuracy, range, interface type (analog, I2C, SPI, UART), power consumption, and environmental tolerance.

Example: For the smart agriculture use-case, the Capacitive Soil Moisture Sensor v1.2 is chosen because it is resistant to corrosion and provides analog output that can be easily read by the Arduino ADC.

3. **Interface the Sensor with Hardware:** Connect the sensor to the main processing unit (e.g., microcontroller or microprocessor). This includes power connections (VCC and GND), signal lines (analog or digital), and optional communication interfaces (I2C, SPI, UART). Proper electrical compatibility (voltage levels) and pin mapping must be ensured.

Example: The capacitive soil moisture sensor is connected to the Arduino Uno via the A0 pin for analog signal input, 5V for power, and GND for grounding.

- 4. Write and Upload the Sensor Code:** Develop firmware to initialize the sensor, read its data, and process the values. Libraries provided by sensor manufacturers can simplify this process. Code is then compiled and uploaded to the microcontroller using an IDE (e.g., Arduino IDE, STM32CubeIDE).

Example: An Arduino sketch is written to read analog values from A0, convert them into soil moisture percentages, and print them to the serial monitor every second.

- 5. Calibrate the Sensor:** Calibration is essential for accuracy. It involves mapping raw sensor data to meaningful physical values by comparing outputs against known standards or reference conditions and adjusting the software accordingly.

Example: The soil moisture sensor is calibrated by placing it in dry and fully wet soil. The analog readings are recorded (e.g., 1023 for dry, 300 for wet), and these values are used to scale output to a 0–100% moisture range.

- 6. Validate and Test the Sensor:** Once calibrated, run field or lab tests to validate if the sensor readings are consistent, accurate, and reliable. Data should be logged, compared with manual readings or ground truth, and analysed for anomalies.

Example: The soil moisture sensor is deployed in a test garden. Manual moisture readings are taken using a probe and compared with sensor data logged over time to validate performance.

- 7. Integrate Sensor Data with Application Logic:** Use the validated data in the main control or decision-making loop of your application. This may involve triggering actuators, sending alerts, or updating cloud databases depending on the system's goals.

Example: When the soil moisture percentage drops below 30%, the Arduino turns on a relay to activate a water pump for irrigation. The action is logged to an SD card and transmitted via Wi-Fi.

- 8. Monitor and Maintain the System:** Even after successful deployment, regular monitoring is essential to catch sensor drift, failures, or environmental damage. Maintenance might include cleaning, recalibration, or firmware updates.

Example: A bi-weekly check is scheduled where moisture sensors are inspected for corrosion, readings are verified with a reference meter, and firmware is updated if performance degrades.

After sensor integration in a Data Acquisition (DAQ) system, the next step is *signal conditioning*. This step is crucial because the raw signals coming from sensors are often not in a form that can be directly read or accurately interpreted by the DAQ hardware.

Signal conditioning involves preparing the signal through processes such as amplification, filtering, isolation, and linearization. Amplification boosts

weak sensor signals to a level suitable for the analog-to-digital converter (ADC). Signal Filtering removes unwanted electrical noise or irrelevant frequency components from the signal. Electrical isolation may be applied to protect the system from voltage spikes or ground loops, especially in industrial environments.

In cases where the sensor output is non-linear, linearization algorithms are used to convert the signal into a format that reflects the actual measured parameter correctly. For example, a thermocouple used to measure temperature produces a small, non-linear voltage signal that must be amplified and linearized before it can be accurately digitized and processed by the DAQ system. The subsequent section relates to the discussion on Signal Conditioning

Check Your Progress 2

1. What does sensor integration involve in a DAQ system?

.....

.....

.....

2. Why is calibration important during sensor integration?

.....

.....

.....

3. How does proper sensor integration benefit system reliability?

.....

.....

.....

4.2.2 Signal Conditioning

In a Data Acquisition (DAQ) system, signal conditioning is the process of modifying and preparing raw signals from sensors so they can be accurately and safely interpreted by the DAQ hardware. When a sensor is integrated into the system, it generates an electrical signal—often in a low-voltage, noisy, or non-linear form—that cannot be directly processed by the DAQ unit. Signal conditioning acts as an essential intermediary step that ensures the sensor's output is compatible with the input requirements of the DAQ device.

This step of DAQ i.e. Signal conditioning is directly linked to sensor integration because it transforms the raw sensor signal into a clean, usable format that the DAQ system can digitize and analyse. Without signal conditioning, the data collected from sensors would likely be inaccurate or even unusable. For example, when a thermocouple is used to measure temperature, the small voltage it produces must be amplified, filtered, and linearized before the DAQ system can reliably interpret it as a temperature reading. Thus, signal conditioning is a vital step in ensuring accurate and

dependable data acquisition after the sensor has been integrated into the system.

This process of Signal conditioning typically involves several key functions, as listed below:

- **Amplification:** Amplification is used to boost weak signals, such as those from thermocouples, to a measurable level.
- **Filtering:** Filtering removes unwanted noise and interference that could distort the signal, like electrical hum from nearby equipment.
- **Electrical isolation:** It is applied in some cases to protect the DAQ system from voltage spikes or ground loops, which can occur when sensors are deployed in electrically noisy or industrial environments.
- **Linearization:** It is the process of converting the non-linear signals into a form that accurately reflects real-world quantities; some sensors produce non-linear outputs that must be linearized—mathematically adjusted—to reflect real-world quantities accurately.
- **Transducing:** In some applications, signal conditioning also involves converting the signal from one form to another, such as from current to voltage or from thermoelectric voltage to a temperature value

Signal conditioning directly follows sensor integration in the DAQ workflow. Once a sensor is physically connected and its output can be read, signal conditioning ensures that this output is usable and reliable for further processing. Sensors often produce raw, low-level, or noisy signals, which cannot be directly fed into the DAQ's ADC or logic circuits without causing errors or damage. Signal conditioning aligns the sensor's output characteristics with the requirements of the DAQ system, making it a critical intermediary step.

Example: When integrating a thermocouple sensor with a DAQ system, the thermocouple produces a very small voltage that also varies non-linearly with temperature. A signal conditioning circuit amplifies this voltage, filters out noise, and linearizes it to produce a signal that the DAQ can read accurately as a temperature value.

Although after signal conditioning the signals can be directly forwarded for Analog-to-Digital conversion (ADC) but many times multiple sensors are involved in data collection, in that case the stage of multiplexing intervenes, where the multiple input signals are combined into a single channel, which can then be read by a single analog-to-digital converter (ADC).

The subsequent section relates to the discussion over the intermediary stage of multiplexing.

☛ **Check Your Progress 3**

1. What is the main purpose of signal conditioning in a DAQ system?

.....

.....

2. List two common signal conditioning techniques.

.....

.....

.....

3. What role does linearization play in signal conditioning?

.....

.....

.....

4.2.3 Multiplexing

Multiplexing and signal conditioning are two distinct but often closely related steps in a Data Acquisition (DAQ) system. They work together to enable efficient handling of signals from multiple sensors using limited hardware resources. The process of Multiplexing is required to combine multiple input signals into a single channel or path, which can then be read by a single analog-to-digital converter (ADC). This is especially useful when a DAQ system needs to acquire data from many sensors, but has only a limited number of ADCs.

In brief, multiplexing allows multiple conditioned signals to share a single ADC, while signal conditioning ensures each signal is in a proper form for accurate measurement. When used together correctly, they make DAQ systems more scalable, cost-effective, and efficient.

Example: Imagine a DAQ system collecting data from 8 temperature sensors using a single ADC. A multiplexer is used to sequentially route one sensor signal at a time to the ADC. Before each signal reaches the Multiplexer (MUX), it passes through its own signal conditioning module to amplify and linearize the signal, ensuring consistent and accurate digital conversion.

In a DAQ system where multiple sensors are used, it's inefficient and costly to assign a separate ADC to each sensor. Instead, a multiplexer (MUX) is used to route multiple analog signals one at a time to a shared ADC. However, before these signals are digitized, they must be cleaned and adjusted for accurate measurement—this is the job of signal conditioning.

There are two main ways to integrate signal conditioning with multiplexing:

1. Per-Channel Signal Conditioning (Before Multiplexing)
2. Shared Signal Conditioning (After Multiplexing)

Per-Channel Signal Conditioning (Before Multiplexing):

When signal conditioning is applied before multiplexing, each sensor channel has its own dedicated signal conditioning circuit. This means that every signal is individually amplified, filtered, and, if necessary, linearized before entering the multiplexer. This approach offers high accuracy and flexibility, especially when dealing with sensors that produce different types of signals or operate over different ranges.

For example, consider a system monitoring eight temperature points using thermocouples. Each thermocouple produces a very low and non-linear voltage, requiring amplification, noise filtering, cold junction compensation, and linearization. In this case, applying signal conditioning before the multiplexer ensures that each thermocouple's output is properly prepared for accurate ADC conversion, despite their signal differences.

Shared Signal Conditioning (After Multiplexing):

When signal conditioning is applied after multiplexing, all sensor outputs are first routed to a multiplexer, which selects one signal at a time to be sent through a shared signal conditioning stage before reaching the ADC. This method is cost-effective and simpler in terms of hardware, but it assumes that all sensors produce similar signal types and levels. It is best suited for systems using identical sensors in similar conditions.

For example, if you are measuring voltage from eight identical pressure transducers that all output 0–5V linear signals, you can safely route their outputs to a multiplexer first. The selected signal can then be passed through a common amplifier and filter setup before being digitized. Since the signal characteristics are consistent across channels, a single conditioning circuit can adequately handle all inputs.

In short, signal conditioning before multiplexing is ideal for mixed or complex sensor configurations where precise and individual processing is needed, while signal conditioning after multiplexing is a practical solution for uniform sensor arrays where a shared conditioning path is sufficient.

After multiplexing, triggering is the next important step of the Data Acquisition process, multiplexing and triggering are both critical concepts and they serve different roles. Multiplexing refers to the process of sequentially sampling multiple input signals using a single analog-to-digital converter (ADC). Whereas the Triggering defines when data acquisition starts or records data. It's a control mechanism to capture meaningful events, rather than recording continuous data.

The subsequent section of this unit relates to the understanding of the triggering and its role in data acquisition.

☛ Check Your Progress 4

1. What is multiplexing in the context of DAQ systems?

.....

.....

.....

2. What is per-channel signal conditioning?

.....

.....

.....

3. When is shared signal conditioning used?

.....
.....
.....

4.2.4 Triggering

In a Data Acquisition (DAQ) system, multiplexing and triggering are two essential functions that work together to efficiently capture and manage input signals. Multiplexing involves sequentially sampling multiple input channels using a single analog-to-digital converter (ADC). Instead of having a separate ADC for each sensor or signal source, a multiplexer (MUX) cycles through each input one at a time, feeding them into a single ADC for conversion into digital values. This approach reduces hardware complexity and cost, especially when dealing with many input signals.

On the other hand, triggering controls when the data acquisition process begins. Rather than recording data continuously, which can be inefficient and lead to unnecessary data storage, triggering ensures that acquisition only starts under specific conditions—such as a threshold voltage being crossed, an external digital signal being received, or a predefined time event occurring. This allows the system to capture meaningful data related to specific events or states.

These two processes are closely related within a DAQ system. A trigger typically initiates the data acquisition sequence, after which the system begins multiplexing through the selected input channels. This coordination ensures that data from multiple sources is captured only when it's relevant, enhancing both efficiency and data quality. In more advanced setups, triggering can also synchronize the MUX cycle with external events, ensuring precise timing and alignment between data channels and real-world processes. In this way, triggering not only governs the start of acquisition but also helps maintain data integrity throughout the multiplexing process.

In any Data Acquisition (DAQ) system, triggering and sampling are closely linked, as triggering determines when the system captures data, while sampling is the process of capturing that data at specific intervals i.e. In short, triggering controls when sampling occurs, allowing you to start data acquisition only when a specific event happens, ensuring the system captures relevant, time-sensitive data.

Without proper triggering, a DAQ system might capture excessive or unnecessary data, leading to large data sets and slower processing times. Triggering allows you to control when to sample, ensuring that only meaningful data is captured, thus improving efficiency and reducing the burden on storage and processing.

The subsequent section relates to the understanding of the sampling and its role in DAQ.

1. What is triggering in DAQ?

.....
.....
.....

2. How does triggering enhance data acquisition?

.....
.....
.....

3. How are triggering and multiplexing related?

.....
.....
.....

4.2.5 Sampling

We learned that Triggering defines the condition that initiates the data acquisition process. The system waits for a specific event or signal (trigger) to occur before starting to record data. This ensures that data is only captured during relevant or meaningful events, rather than continuously or randomly, which helps in managing storage and focusing on important data points.

Sampling is the process by which the system captures data from an input signal at discrete intervals. The system samples the input signal at a constant rate or when specific conditions (such as a trigger) are met. This process converts the analog signal into digital data for processing or analysis.

Thus, Sampling refers to the process of measuring the analog signal's voltage at specific time intervals, known as the sampling rate. This is necessary because analog signals are continuous in time and value, while digital systems require discrete data points. By taking snapshots of the signal at regular intervals, the system captures enough information to digitally represent the original analog signal for processing, storage, or display.

In a Data Acquisition (DAQ) system, sampling is a critical step that takes place after multiplexing and before analog-to-digital conversion. Multiplexing is used when the system needs to acquire data from multiple sensors but has only one analog-to-digital converter (ADC). The multiplexer acts as a switch, selecting one sensor signal at a time and routing it to the ADC. Once a specific sensor channel has been selected by the multiplexer, the signal from that sensor is then sampled by the ADC.

Since only one signal can be sampled at any given moment, sampling must occur after the multiplexer selects a signal. The process typically happens in rapid succession: the multiplexer selects channel 1, the ADC samples and converts that signal, then the multiplexer switches to channel 2, and the process repeats. This method, known as time-division multiplexing, allows

the system to cycle through multiple channels quickly enough that it gives the appearance of simultaneous data collection, even though the channels are being read sequentially.

For example, consider a DAQ system connected to four temperature sensors. If the system operates at a total sampling rate of 1000 samples per second, and it uses a multiplexer to handle all four inputs, each sensor might be sampled at 250 samples per second. The multiplexer cycles through the four channels, allowing the ADC to sample and digitize each one in turn. Despite using only one ADC, the system efficiently captures data from multiple sources by coordinating multiplexing and sampling.

In short, sampling always follows multiplexing in a DAQ system that uses a shared ADC. The selected signal from the multiplexer is sampled to capture its value at that instant, allowing the ADC to convert it into a digital form for further processing. This structured sequence ensures accurate, time-synchronized measurements from multiple sensors using minimal hardware. i.e. the stage of sampling in the DAQ process acts as the bridge between multiplexing and ADC. It ensures that the analog signal selected by the multiplexer is accurately captured in time before being converted into a digital format. Without sampling, the transition from a chosen analog signal to its digital representation would not be possible. It preserves the temporal and voltage accuracy needed for reliable data acquisition in systems that rely on shared conversion resources.

The next step after sampling is the Analog-to-Digital Conversion (ADC), where the system transforms the sampled analog voltage into a digital form that can be processed, visualized, or logged by the system. The subsequent section relates to the Analog-to-Digital Conversion (ADC) stage of the DAQ process.

Check Your Progress 6

1. What is the function of sampling in DAQ?

.....

.....

.....

2. Why must sampling follow the Nyquist criterion?

.....

.....

.....

3. How does sampling relate to multiplexing and ADC?

.....

.....

.....

.....

4.2.6 Analog-to-Digital Conversion (ADC)

In a Data Acquisition System (DAQ), the ADC (Analog-to-Digital Conversion) stage plays a critical role in converting continuous analog signals into discrete digital data that a computer or microcontroller can process. The ADC stage allows the system to interface with real-world analog signals such as temperature, pressure, voltage, or current. This is necessary for monitoring, processing, and analysing physical phenomena in various applications, such as industrial automation, scientific research, or instrumentation.

The ADC process typically involves several stages: *signal conditioning, sampling, hold, quantization, conversion, encoding, and output*, each with a specific role in converting the analog signal to a digital format. Below, a brief discussion over the stages of ADC is given, with suitable examples for understanding each stage.

1. **Signal Conditioning:** Before the analog signal reaches the ADC, it often needs to be conditioned. Signal conditioning typically involves amplifying, filtering, or isolating the signal to ensure that it falls within the input range of the ADC and is free from noise. This step is crucial because most sensors output weak or noisy signals that cannot be directly fed to the ADC. **Example:** Consider a thermocouple used to measure temperature. The voltage generated by the thermocouple is often in the millivolt range, which may be too small for the ADC to accurately detect. To resolve this, a signal amplifier boosts the signal to a more usable level, such as from 1 mV to a range that is within the ADC's input specifications (e.g., 0-5V). Additionally, a low-pass filter may be applied to eliminate high-frequency noise from the signal.

Note: Multiplexing and Triggering are performed on requirement basis only

2. **Sampling:** Sampling is the process of taking discrete measurements of the continuous analog signal at specific time intervals. The rate at which the signal is sampled is known as the sampling rate or frequency. According to the Nyquist-Shannon sampling theorem, the sampling rate must be at least twice the highest frequency of the analog signal to accurately represent it digitally. **Example:** Let's assume you are measuring an analog sine wave with a frequency of 10 Hz. The ADC samples this signal at a rate of 100 Hz, meaning it takes 100 samples per second. Since the sampling rate (100 Hz) is higher than twice the frequency of the signal (20 Hz), the digital representation of the sine wave will be accurate.
3. **Hold Stage (Sample-and-Hold):** Once the analog signal is sampled, it needs to be "held" steady long enough for the ADC to perform its conversion. The sample-and-hold circuit stores the sampled voltage value for a brief moment to allow the ADC enough time to convert it into a digital format. **Example:** If you are sampling a fast-changing signal, like a fluctuating current, the sample-and-hold circuit ensures that the ADC can "lock" the value for a short time. Without this, the ADC might be unable to capture a stable value, resulting in inaccuracies. For

instance, if the signal changes rapidly, the ADC could sample at an incorrect point in time without the hold mechanism.

4. **Quantization:** Quantization is the process of converting the continuous amplitude of the analog signal into discrete levels. The precision of this conversion depends on the resolution of the ADC, which is typically expressed in bits. A higher resolution means more discrete levels and greater precision in representing the analog signal. **Example:** Consider an 8-bit ADC with a reference voltage of 5V. The 8-bit resolution allows for 256 discrete levels ($2^8 = 256$). The quantization process divides the input range (0 to 5V) into 256 equally spaced steps, with each step representing a specific voltage level. For instance, an input signal of 2.8V might correspond to one of the quantization levels, like level 178 (out of 256 possible levels). However, the exact analog value could fall between two levels, so quantization introduces a small error called quantization error.
5. **Conversion:** The conversion process is where the sampled and quantized analog value is transformed into a digital value. Different ADC architectures, such as Successive Approximation Register (SAR), Flash, or Delta-Sigma, achieve this conversion in different ways. The ADC compares the sampled signal to a reference value and uses an algorithm to output a binary representation of the closest matching digital value. **Example:** In a SAR ADC, the conversion process works by comparing the input signal to the reference voltage using a binary search method. Suppose the input signal is 2.8V, and the reference voltage is 5V. The ADC starts by setting the most significant bit (MSB) to "1" and compares the input to this value. The process continues bit by bit until the best approximation of the input signal is achieved, resulting in a binary output like 11001010 (in the case of an 8-bit ADC).
6. **Encoding (Digital Output):** After the conversion, the resulting digital value is encoded into a binary number that represents the analog signal in digital form. This is the final output of the ADC stage. The encoded value is typically a binary code, ready to be processed or transmitted by the digital system. **Example:** Let's assume the ADC has converted the input signal of 2.8V into a digital value of 178 using an 8-bit resolution. This value is encoded as the 8-bit binary number 10110010. This binary value is then sent to the microcontroller or computer for further processing.
7. **Output and Processing:** The final step in the ADC process is sending the digital output to the processing system, such as a microcontroller, computer, or other digital processing unit. The digital data can be used for various purposes, such as control, analysis, or visualization. **Example:** In a temperature control system, the microcontroller receives the digital temperature reading (e.g., 178 from the ADC) and processes it to determine if the furnace needs to be adjusted. If the reading is too low, the microcontroller might activate the heating element to raise the temperature. The digital data from the ADC enables the system to make precise and timely decisions based on real-world analog inputs.

Let's summarize and understand the actual process of ADC through an example of a temperature measurement system using a thermocouple connected to a DAQ system (as discussed above). The thermocouple generates a small voltage that corresponds to the temperature, and this voltage must be converted into a digital value for further processing.

Here's how the ADC works in this setup:

1. **Signal Conditioning:** The raw voltage from the sensor is amplified and filtered to remove noise, ensuring it's within the input range for the ADC.
2. **Sampling:** The system samples the voltage at a rate of 1 kHz to capture fast changes in temperature.
3. **Hold Stage:** A sample-and-hold circuit stores each sample briefly for the ADC to process it.
4. **Quantization:** The ADC quantizes the signal into 256 levels if it's an 8-bit ADC.
5. **Conversion:** The ADC converts the voltage into a binary value. For example, a voltage of 2.5V might correspond to a digital value of 128 in an 8-bit system.
6. **Encoding:** The 128 is the digital representation of 2.5V, and this value is sent to the microcontroller.
7. **Output and Processing:** The microcontroller receives the digital value and uses it to adjust the furnace's temperature control system accordingly.

The ADC first converts the continuous analog signal into a digital format by sampling, quantizing, encoding, and producing a binary output. After the ADC has converted the signal, the resulting digital data may undergo further data processing. For instance, it may be scaled, filtered, or interpreted based on the system's requirements. The subsequent section relates to the discussion on the data processing stage of the DAQ process.

Check Your Progress 7

1. What is the primary role of an ADC in DAQ?

.....

.....

.....

.....

2. What is quantization in ADC?

.....

.....

.....

.....

.....

3. What does a sample-and-hold circuit do?

.....
.....
.....

4.2.7 Data Processing

Data processing in the ADC (Analog-to-Digital Conversion) process refers to the steps involved in transforming the sampled, quantized, and encoded data into a usable digital output that can be interpreted or further analysed by digital systems.

Once conditioned signal, i.e. the analog signal is passed through an Analog-to-Digital Converter (ADC). It passes through the stages of sampling, quantization, and encoding; thereafter the signal is digitized.

The data processing involves the following:

1. **Validation and Error Detection** (Ensures data integrity and flags any outliers or errors): After the signal is digitized, the system performs data validation to ensure accuracy and reliability. This may involve range checking to identify values that fall outside expected limits and error detection to flag or discard corrupt or incorrect data. In addition, data smoothing techniques like moving averages may be applied to remove erratic spikes or fluctuations. For example, if a sudden spike shows a temperature of 1000°C—far beyond the sensor’s capability—it may be recognized as an error and either flagged or corrected using previous valid readings.
2. **Calibration and Transformation** (Converts raw data into meaningful physical measurements): The Digitized values from the ADC often require calibration and transformation to convert them into real-world units. Calibration corrects discrepancies between the sensor’s output and actual physical values, ensuring the data reflects true measurements. In many cases, linearization is necessary, especially if the sensor’s response is nonlinear. For instance, if 0V represents 0°C and 5V represents 100°C, a digital value of 150 (from an 8-bit ADC) might be converted into a temperature reading of 58.8°C using a formula or lookup table.
3. **Aggregation and Filtering** (Combines data from multiple sources and removes noise):

When data is collected from multiple sensors, data aggregation is used to combine readings for unified analysis. This might involve calculating averages or weighted values. At the same time, digital filtering techniques such as low-pass filters or Kalman filters are used to reduce high-frequency noise or smooth the data. In a weather station system, for example, temperature data from multiple locations can be averaged and filtered to provide a stable and representative reading of ambient conditions.

This stage i.e. the data processing stage in a Data Acquisition (DAQ) system is a critical phase where raw digital data collected from sensors is refined, interpreted, and transformed into usable and meaningful information. This step follows the analog-to-digital conversion (ADC) and ensures that the digitized signal can be utilized effectively for monitoring, control, or analysis.

☛ Check Your Progress 8

1. What is the first step in data processing after digitization?

.....

.....

.....

2. Why is calibration important in data processing?

.....

.....

.....

3. What is data aggregation in this context?

.....

.....

.....

4.2.8 Data Logging

In a Data Acquisition (DAQ) process, data logging and data processing are two closely related but distinct stages that work together to transform raw sensor readings into meaningful information. Data logging refers to the act of recording raw data collected from sensors or measurement devices over time. This data is typically stored in formats such as CSV files, binary files, or databases, and the primary goal is to ensure reliable storage for future retrieval, analysis, or archival purposes. Data logging often takes place in real time as the data is being acquired.

Data processing, on the other hand, involves analysing, filtering, converting, or otherwise transforming the raw data into a more usable and interpretable form. This can include tasks such as applying filters to remove noise, converting units, calculating statistical measures, or detecting anomalies. Data processing can occur either in real time (as data is being acquired) or offline (after the data has been logged).

The relationship between data logging and data processing can follow different workflows. In some systems, data is acquired and logged first, and then processed later in an offline setting. In other systems, data may be processed in real time and then logged in a filtered or compressed form. In both cases, data logging supports data processing by preserving the raw information, while data processing can enhance data logging by improving the quality or reducing the volume of the stored data. For example, in a

temperature monitoring system, sensor readings might be logged every second and then processed later to calculate hourly averages and identify long-term trends.

☛ Check Your Progress 9

1. What is the purpose of data logging?

.....

.....

.....

2. How does data logging relate to data processing?

.....

.....

.....

3. Can data logging and processing happen simultaneously?

.....

.....

.....

4.2.9 Display/Control/Transmission

Display, control, and transmission are integral parts of a DAQ system that interact with data logging to provide real-time feedback, enable system management, and support remote communication.

- **Display** refers to the real-time visualization of acquired data through graphical interfaces, charts, dashboards, or numeric readouts. This data is typically sourced directly from the DAQ hardware or from the data being logged. Real-time display often works in parallel with data logging, showing users live values while simultaneously recording them for future use.
- **Control** involves using acquired data to make decisions or trigger actions within the system, such as turning devices on or off or adjusting parameters based on preset conditions. This control logic may rely on real-time data or on trends and thresholds derived from previously logged data. For instance, if a temperature reading logged over time shows an upward trend beyond a safe limit, the system may automatically activate cooling equipment.
- **Transmission** refers to sending data to remote systems, databases, or cloud platforms for further analysis, archiving, or monitoring. Logged data is often packaged and transmitted at regular intervals or on-demand, ensuring that off-site users or systems have access to historical and real-time data.

Together, these components form a cohesive workflow: data is acquired and logged, while display provides immediate feedback, control acts on

conditions derived from the data, and transmission extends accessibility of the logged information. Data logging thus acts as the backbone, supporting and integrating with each of these operations to ensure accurate recordkeeping, timely responses, and system-wide

☛ Check Your Progress 10

1. What does the display function do in a DAQ system?

.....

.....

.....

2. What is the role of control in this context?

.....

.....

.....

3. How is data transmission handled in DAQ?

.....

.....

.....

4.3 DAC (DIGITAL-TO-ANALOG CONVERSION)

A Digital-to-Analog Converter (DAC) is an electronic device or circuit that converts digital data—usually in binary form—into a corresponding analog voltage or current. This is essential in systems where digital controllers (like microcontrollers, computers, or DAQs) need to communicate or control analog devices such as speakers, motors, or sensors.

Steps Involved in the DAC Process are :

1. **Receive Digital Input:** The DAC receives a binary digital value, which is typically provided by a microcontroller, processor, or DAQ system. This digital value represents the desired output, which could be a specific voltage or current level. For example, a 10-bit DAC might receive a value like 0011011010 in binary.
2. **Decode the Digital Word:** Once the digital input is received, the DAC decodes the binary value based on its resolution (e.g., 8-bit, 10-bit, or 12-bit). The resolution determines how finely the DAC can represent the input, and it calculates the corresponding output level based on a reference voltage or current. In a 10-bit DAC, the binary value might range from 0 to 1023, with each value corresponding to a specific voltage level.
3. **Generate Analog Output (Stepped Voltage/Current):** After decoding the digital word, the DAC uses internal circuitry—such as resistor ladders, current sources, or charge-scaling techniques—to convert the

digital value into a stepped analog signal. The output is typically a voltage or current that approximates the value represented by the digital input. The steps between these values are proportional to the DAC's resolution, meaning the higher the resolution, the finer the steps.

4. **Apply Smoothing (Optional):** In many cases, the DAC output may resemble a series of discrete steps rather than a smooth curve. To convert this into a continuous waveform, a low-pass filter (usually an RC filter) is applied to smooth the output. This filter helps to reduce the sharp transitions between the steps and results in a more continuous analog signal.
5. **Output the Analog Signal:** Finally, the smoothed or stepped analog signal is output from the DAC, ready to be used by the system. This analog signal can be sent to external devices, such as audio equipment, motors, or display systems, to control or interact with them. The output is typically in the form of a voltage or current that can be further amplified or processed as needed.

Each of these steps ensures that a digital input can be accurately transformed into a corresponding analog signal, which is vital for various applications like audio playback, signal generation, and control systems.

A DAC (Digital-to-Analog Converter) can be useful in a DAQ (Data Acquisition) system, depending on the specific application. While most DAQ systems are primarily designed to measure analog signals—such as voltage, current, or temperature—and convert them into digital data using an ADC (Analog-to-Digital Converter), there are many situations where a DAC adds valuable functionality.

A DAC becomes particularly useful when the system needs to output analog signals. For example, in applications involving control of actuators, motors, or valves, the DAQ system may need to generate an analog control signal, which requires a DAC. In closed-loop control systems, the DAQ might read data from sensors using an ADC, process that data, and then output a corresponding analog control signal via the DAC to influence the system's behavior—such as adjusting a heater, regulating a pump, or steering a robotic arm.

DACs are also important in test and simulation scenarios, where the DAQ system must generate analog signals to simulate sensor outputs or test analog circuitry. In such cases, the ability to produce precise analog waveforms allows the DAQ system to mimic real-world conditions for validation or troubleshooting.

In short, a DAC is not always a required component in a DAQ system, it becomes essential in applications that involve analog output, control, or simulation, complementing the input-focused role of ADCs.

1. When is DAC used in a DAQ system?

.....

.....

.....

2. What does a DAC's resolution determine?

.....

.....

.....

3. Why is smoothing sometimes required after DAC?

.....

.....

.....

4.4 HIGH SPEED DAQ

A High-Speed Data Acquisition (DAQ) system refers to a setup designed to sample and record data at extremely fast rates. These systems are equipped with high-speed analog-to-digital converters (ADC), sensors, and signal conditioning equipment capable of capturing high-frequency signals with high precision.

The High-speed DAQ systems are fundamental in applications where rapid data acquisition is necessary for accurate analysis, decision-making, and model training in **data science**. These systems enable real-time monitoring, precise signal measurement, and the handling of vast amounts of high-frequency data, making them indispensable in fields like engineering, healthcare, finance, and scientific research.

The key characteristic of high-speed DAQ systems is their sampling rate, which is often in the range of kilohertz (kHz) to gigahertz (GHz). This allows them to measure dynamic signals and capture transient events that occur over very short periods. A brief description of the Key Features of High-Speed DAQ is given below:

- **High Sampling Rate:** Capable of sampling signals at very high frequencies (e.g., millions or billions of samples per second).
- **Wide Bandwidth:** Able to capture signals over a broad frequency range, from DC to high-frequency signals (up to GHz).
- **High Resolution:** Ensures precise measurements with fine granularity, typically 12-bit, 16-bit, or higher.
- **Low Latency:** Capable of processing and transmitting data with minimal delay, essential for real-time applications

High-speed DAQ systems play a crucial role in **data science**, especially in fields where capturing high-frequency signals, real-time data, or transient events is critical. Here's why they are essential:

1. **Real-Time Data Collection:** High-speed DAQ systems allow for the **real-time collection** of data from fast-moving processes. In applications like **electrical engineering**, **telecommunications**, and **industrial monitoring**, real-time data is essential for making instant decisions, troubleshooting, or controlling systems.
2. **Time-Series Analysis:** In data science, time-series data is a crucial type of data used to analyze how systems behave over time. High-speed DAQs enable the collection of **high-frequency time-series data**, which can be used for tasks like **predictive maintenance**, **anomaly detection**, and **trend analysis**.
3. **Data Acquisition for Complex Simulations:** High-speed DAQs are commonly used to feed **simulation models** with data, where accurate and timely input is required. For example, in **financial modeling**, **engineering simulations**, or **scientific research**, where rapid or continuous data needs to be processed and analyzed at high speeds.
4. **Signal Integrity in Noise-Intensive Environments:** High-speed DAQs, particularly those with **high resolution**, are designed to accurately capture signals even in environments with high levels of **electrical noise**. This is important for applications such as **signal processing** in telecommunications or **medical instrumentation** (e.g., capturing data from ECG, EEG).
5. **Data-Intensive Applications:** High-speed DAQs are capable of handling large volumes of data from multiple channels. In **data science**, this means the ability to capture complex datasets with high fidelity for tasks such as **machine learning**, **statistical modeling**, or **image processing** in real-time.
6. **Testing and Quality Control:** In industrial and manufacturing settings, high-speed DAQ systems are used for **quality control** and **performance testing**. The ability to capture fast, transient events allows for thorough analysis of product behavior under various conditions, leading to **improved quality assurance** and **predictive maintenance**.
7. **Advanced Signal Processing:** High-speed DAQs feed raw data into data science pipelines, where **advanced signal processing** techniques, such as **Fourier transforms** or **wavelet analysis**, can be applied. These techniques are crucial for filtering, compressing, or enhancing the data for use in algorithms and machine learning models.

Examples of High-Speed DAQ in Data Science Applications:

1. **Financial Market Analysis:** In high-frequency trading, high-speed DAQ systems are used to capture market data at millisecond or microsecond intervals to make real-time trading decisions based on rapid market movements.

2. **Healthcare and Biomedical Engineering:** High-speed DAQ systems are used in medical devices such as **ECGs** or **EEGs** to capture and analyze bio-signals at high frequencies for diagnostic purposes or real-time monitoring of patients.
3. **Engineering and Physics Research:** In experimental physics or engineering, high-speed DAQ systems are used to measure transient events, such as shock waves, vibration patterns, or electrical signals in real-time for further analysis in simulations or models.
4. **Autonomous Vehicles:** In autonomous vehicle development, high-speed DAQ systems are used to collect data from sensors (such as LIDAR, cameras, and radar) at high rates to enable real-time decision-making and machine learning for navigation and safety.

🔧 Check Your Progress 12

1. What defines a high-speed DAQ system?

.....

.....

.....

2. Name one application of high-speed DAQ in healthcare.

.....

.....

.....

3. Why is high resolution important in high-speed DAQs?

.....

.....

.....

4.5 SUMMARY

This unit provided a comprehensive overview of the entire data acquisition process used in digital systems to collect, convert, and process real-world physical signals. It begins by emphasizing the importance of accurate data collection for data science applications and outlines the key stages of a DAQ system, starting with sensor integration, where physical phenomena like temperature or pressure are converted into electrical signals. This is followed by signal conditioning, which prepares these signals through amplification and filtering to ensure compatibility with the system's analog-to-digital converter (ADC). In multi-sensor systems, multiplexing is used to channel several inputs through a single ADC, while triggering controls when data collection starts, ensuring relevant and efficient data capture.

Subsequent stages include sampling, where signals are captured at regular intervals, and analog-to-digital conversion, which transforms these sampled signals into digital data for processing. The data processing stage involves

error checking, calibration, filtering, and transformation into meaningful values, followed by data logging for storage and future analysis. The document also discusses how this data can be displayed in real time, used for control actions, or transmitted to remote systems. Additionally, it highlights the role of DAC (Digital-to-Analog Conversion) in systems requiring analog output and introduces high-speed DAQ systems that operate at high sampling rates for applications like biomedical monitoring, industrial automation, and real-time analytics. Overall, the unit builds a clear understanding of how physical data is acquired, digitized, processed, and utilized in intelligent digital systems.

4.6 SOLUTIONS/ANSWERS

☛ Check Your Progress 1

1. What is data acquisition in the context of data science?

ANSWER Data acquisition refers to the process of collecting data from various sources for analysis. The success of data science projects depends heavily on the quality and accuracy of this data.

2. What types of sources can data be collected from in data acquisition?

ANSWER Data can be collected from sensors, web scraping, APIs, surveys, and existing databases.

3. What is the main purpose of the Data Acquisition (DAQ) process?

The DAQ process is used to collect and measure physical phenomena—like temperature, pressure, voltage, or motion—and convert them into digital data that can be analyzed by a computer.

4. Why is signal conditioning necessary before sampling in a DAQ system?

ANSWER Signal conditioning prepares the raw sensor output by amplifying, filtering, and converting it to make it compatible with the Analog-to-Digital Converter (ADC), ensuring accurate sampling and processing.

5. What is the role of triggering in the data acquisition process?

ANSWER Triggering ensures that data recording begins only when specific conditions are met, such as a certain voltage level or time instance, allowing precise capture of important events

☛ Check Your Progress 2

1. What does sensor integration involve in a DAQ system?

It involves the physical connection and logical configuration of sensors with DAQ hardware and software to ensure accurate data collection.

2. Why is calibration important during sensor integration?

ANSWER Calibration ensures that sensor outputs correspond accurately to real-world values, minimizing errors.

3. How does proper sensor integration benefit system reliability?

It minimizes noise, interference, and data loss, thus making the DAQ

system more robust.

☛ Check Your Progress 3

1. **What is the main purpose of signal conditioning in a DAQ system?**
ANSWER To modify raw sensor signals so they can be accurately interpreted by the DAQ hardware.
2. **List two common signal conditioning techniques.**
ANSWER Amplification and filtering.
3. **What role does linearization play in signal conditioning?**
ANSWER It converts non-linear sensor signals into a form that accurately reflects real-world measurements.

☛ Check Your Progress 4

1. **What is multiplexing in the context of DAQ systems?**
ANSWER It refers to the process of routing multiple sensor signals through a single ADC sequentially.
2. **What is per-channel signal conditioning? It applies signal conditioning before multiplexing, allowing each sensor to be individually prepared for ADC.**
3. **When is shared signal conditioning used?**
ANSWER It is used when all sensors produce similar signal types, allowing conditioning after multiplexing.

☛ Check Your Progress 5

1. **What is triggering in DAQ?**
ANSWER Triggering is a control mechanism that starts data acquisition only when specific conditions are met.
2. **How does triggering enhance data acquisition?**
ANSWER It ensures only relevant data is captured, reducing unnecessary storage and improving efficiency.
3. **How are triggering and multiplexing related?**
ANSWER Triggering initiates the acquisition, after which multiplexing can sequence through input channels.

☛ Check Your Progress 6

1. **What is the function of sampling in DAQ?**
ANSWER Sampling captures analog signal values at discrete time intervals, enabling digital representation.
2. **Why must sampling follow the Nyquist criterion?**
ANSWER To avoid aliasing and ensure accurate digital representation of the original signal.
3. **How does sampling relate to multiplexing and ADC?**
ANSWER It occurs after the multiplexer selects a signal and before it is digitized by the ADC.

☞ Check Your Progress 7

1. **What is the primary role of an ADC in DAQ?**
ANSWER To convert continuous analog signals into discrete digital data for processing.
2. **What is quantization in ADC?**
ANSWER It is the process of mapping a range of analog values to discrete digital levels based on the ADC's resolution.
3. **What does a sample-and-hold circuit do?**
ANSWER It holds a sampled analog signal steady during the ADC's conversion process.

☞ Check Your Progress 8

1. **What is the first step in data processing after digitization?**
ANSWER Validation and error detection to ensure data integrity.
2. **Why is calibration important in data processing?**
ANSWER It aligns digitized data with real-world measurements, ensuring meaningful results.
3. **What is data aggregation in this context?**
ANSWER Combining data from multiple sources for unified analysis, often using averaging or filtering.

☞ Check Your Progress 9

1. **What is the purpose of data logging?**
ANSWER To record raw or processed data for future analysis or archival purposes.
2. **How does data logging relate to data processing?**
ANSWER Logging stores the data, while processing interprets or transforms it for meaningful use.
3. **Can data logging and processing happen simultaneously?**
ANSWER Yes, they can occur in real time or separately in offline settings.

☞ Check Your Progress 10

1. **What does the display function do in a DAQ system?**
ANSWER It visualizes real-time data using charts, dashboards, or readouts for monitoring.
2. **What is the role of control in this context?**
ANSWER It uses data to make system decisions or trigger actions like turning devices on/off.
3. **How is data transmission handled in DAQ?**
ANSWER It sends data to remote systems or cloud platforms for remote access and analysis.

☛ Check Your Progress 11

1. When is DAC used in a DAQ system?

ANSWER When digital output needs to be converted back to analog signals for control or simulation purposes.

2. What does a DAC's resolution determine?

ANSWER The granularity of the analog output, with higher resolution offering finer output levels.

3. Why is smoothing sometimes required after DAC?

ANSWER To convert stepped outputs into smooth waveforms using low-pass filters.

☛ Check Your Progress 12

1. What defines a high-speed DAQ system?

ANSWER Its ability to sample and record data at very high rates, often in kHz to GHz ranges.

2. Name one application of high-speed DAQ in healthcare.

ANSWER Capturing real-time ECG or EEG signals for diagnosis or monitoring.

3. Why is high resolution important in high-speed DAQs?

ANSWER To ensure precise measurement of fast-changing signals, especially in noisy environments.

4.7 FURTHER READINGS

1. Practical Data Acquisition for Instrumentation and Control Systems, John Park & Steve Mackay, 1st Edition, Newnes (Elsevier), 2003.
2. Data Acquisition Systems: From Fundamentals to Applied Design, Maurizio Di Paolo Emilio, 1st Edition, Springer, 2013.
3. Measurement and Data Analysis for Engineering and Science, Patrick F. Dunn, 3rd Edition, CRC Press, 2014.
4. Programmable Logic Controllers: Principles and Applications, John W. Webb & Ronald A. Reis, 5th Edition, Prentice Hall, 2002.

